Course Code	:	CYS201
Course Name	:	Modern Cryptography
Lecturer	:	Iftekhar Salam
Academic Session	:	2025/04
Assessment Title	:	Implementation and Analysis of a Secure Encrypted Storage System using Grain-128AEADv2
Submission Due Date	:	1 July 2025, 11:59 PM

Prepared by :

Student ID	Student Name
CYS2309276	Stefanie

Date Received :

Feedback from Lecturer:

Mark:

Own Work Declaration

I acknowledge that my work will be examined for plagiarism or any other form of academic misconduct, and that the digital copy of the work may be retained for future comparisons.

I confirm that all sources cited in this work have been correctly acknowledged and that I fully understand the serious consequences of any intentional or unintentional academic misconduct.

In addition, I affirm that this submission does not contain any materials generated by AI tools, including direct copying and pasting of text or paraphrasing. This work is original creation, and it has not been based on any work of other students (past or present), nor has it been previously submitted to any other course or institution.

Signature: 

Date: 30 June 2025

Table of Contents

Table of Contents **2**

1. Introduction **3**

2. Testing and Evaluation **4**

2.1 Output of Test Vectors 4

2.2 Test Scenarios 4

A. Scenario 1 (Encryption Functionality): 4

B. Scenario 2 (Decryption Functionality): 6

C. Scenario 3 (Decryption with Wrong Input): 7

D. Scenario 4: Avalanche Effect Test 8

E. Scenario 5: Input Edge Cases 10

F. Scenario 6: Encryption/Decryption with wrong input (Key/IV) length 12

G. Scenario 7: Determinism Test 14

3. Construction Analysis **15**

1. Justification for 512 Initialization Rounds in Grain-128AEADv2 15

Why Reducing the Initialization Rounds is Insecure 16

Reference Example 17

2. Rationale for Reintroducing the Key During Clocks 320 to 383 20

Appendix A: Source Code **22**

1. main.py 22

2. gui.py 49

Appendix B: Marking Rubric **81**

Appendix C: Turnitin Summary **82**

Appendix D: Digital Receipt **83**

1. Introduction

The advancement of cryptography in the modern world has led to the continuous development of lightweight ciphers for IoT and embedded systems. Among them is the grainAEADv2 stream cipher that is one of the finalists in the NIST lightweight cryptography standardization process. It offers encryption and authentication for data and design to ensure the confidentiality and integrity of resources in a constrained environment.

This project focuses on implementing the confidentiality or the encryption component of the grain128AEADv2, without incorporating the authentication function (AEAD). The implementation includes key and IV management, encryption, decryption, and testing. This implementation does not include generating or verifying of authentication tag, which is the ACC and REG components.

The goal of implementing the encryption component of the cipher is to test and validate its mechanism. As such, testing vectors and studying components will be highlighted in this report. Additionally, a GUI (Graphical User Interface) was developed to ensure easy user interaction and usability of the cipher's cryptographic abilities.

Additional modification was made to further improve the security of the cipher, which is password-protected encryption. The user can input their password for their associated ciphertext. The feature was developed with the help of SHA256 hashing, which provides a bit mask to XOR with the chosen ciphertext. The bit mask was generated using the user-imputed password. Hence, increasing the security of the ciphers and the corresponding ciphertext.

In this implementation, 2 main python file is made, which is main.py and gui.py. The main.py provide the cipher's main logic and also command line interface, while gui.py will serve function as the main user interaction. To utilize the cipher, run the gui.py in the same folder as main.py. Inside the gui, there will be 3 tab; setup key and IV, encryption, and decryption. User are able to explore its feature and encrypt or decrypt after loading the key and IV.

2. Testing and Evaluation

2.1 Output of Test Vectors

Input	After Loading	After Initialization	Ciphertext
Key: 0x00000000000000000000000000000000 IV: 0x00000000000000000000000 Plaintext: This is a secret test message. Your implementation should be able to handle variable length input messages, and the inputs should not be hardcoded.	NFSR: 00000000000000000000000000000000 LFSR: 00000000000000000000000FFFFFFF7	NFSR: 81F7E0C655D035823310C278438DBC20 LFSR: 8F395A9421B0963364E2ED30679C8EE1	63E8825E421DDF0C8B946209E692775F32A5AB1DF10CF0762697E1E72A5AE30A3E3C8EE345C58640300EE8049E5F07EC29E99E23B95708ECE8D9EF6A3DF895B915CA24451484AE82B15E8F504D05F7389524CB1344FFBDCF7CCF115A4D4BAAF4C000A6F4A5293C6D538D9AF743DB557801892273BB3B2B80998917071E4E3EFEB591F5653779138E09D072605BB0825D033E7F
Key: 0x000102030405060708090a0b0c0d0e0f IV: 0x000102030405060708090a0b Plaintext: This is not a secret message.	NFSR: 000102030405060708090A0B0C0D0E0F LFSR: 000102030405060708090A0BFFFFFF7F	NFSR: B3C2E1B1EEC1F08C2D6EAE957F6AF9D0 LFSR: 0E1F950D45E05087C4CD63FD00EAB310	BE490FF62596FC905FF9A49FC58A6793DEF2D14DC1E7F3FEDA1FD823E8
Key: 0x0123456789abcdef123456789abcdef0 IV: 0x000102030405060708090a0b Plaintext: This is a secret message.	NFSR: 0123456789ABCDEF123456789ABCDEF0 LFSR: 000102030405060708090A0BFFFFFF7F	NFSR: 5458C7F54BAAE53220D764AEF9CF9493 LFSR: 3722B955274CB56559D2430E017E0381	207DCDBD7419905A2F60D9668D49D56F89C459327B97FC9547

2.2 Test Scenarios

A. Scenario 1 (Encryption Functionality):

These test cases are done to ensure the encryption abilities of the cipher. If the encryption algorithm is implemented accurately, the result should provide a corresponding

ciphertext for each key and IV input. A distinct ciphertext should be produced for different IVs or keys.

1. Encrypting a plaintext *P* with a key *K* and initialisation vector *V* to produce the ciphertext *C*.
    - **Key**: 0x0000000000000000000000000000000000
    - **IV**: 0x00000000000000000000000000
    - **Plaintext**: This is a secret test message.
    - **Ciphertext**: 63E8825E421DDF0C8B946209E692775F32A5AB1DF10CF0762697E1E72A5A
2. Encrypting a plaintext *P* with a different key *K′* and initialisation vector *V* to produce the ciphertext *C′*, where $K \neq K'$.
    - **Key**: 0x11111111111111111111111111111111
    - **IV:** 0x00000000000000000000000000
    - **Plaintext**: This is a secret test message.
    - **Ciphertext**: C9C5C1F5B599289A70761E60D0A4EA69DB41ABC40FE7E1EF38302F7009C2
3. Encrypting a plaintext *P* with a key *K* and a different initialisation vector *V′* to produce the ciphertext *C″*, where $V \neq V'$.
    - **Key**: 0x0000000000000000000000000000000000
    - **IV**: 0x11111111111111111111111111
    - **Plaintext**: This is a secret test message.
    - **Ciphertext**: 924BAD0C01B649365FA80F8FF5611BB81EEE0C6006D5DA7E7B91AB63CC76

Based on the ciphertexts, the encryption is functioning correctly, it gives an encryption output, and changing the key or the IV results in a completely different ciphertext. These result passes the functional encryption tests, supporting that the stream cipher is well-constructed.

B. Scenario 2 (Decryption Functionality):

The scenario aims to validate the decryption part of the cipher. For a stream cipher, decryption is performed by generating the same keystream and XORing the keystream with the ciphertext. Therefore, providing the correct key and IV, if the algorithm is correct, then the plaintext should be recovered.

1. Decrypting the ciphertext C with key K and initialisation vector V to recover the plaintext P.
    - **Key**: 0x00000000000000000000000000000000
    - **IV**: 0x000000000000000000000000
    - **Ciphertext**: 63E8825E421DDF0C8B946209E692775F32A5AB1DF10CF0762697E1E72A5A
    - **Plaintext**: This is a secret test message.
2. Decrypting the ciphertext C' with key K' and initialisation vector V to recover the plaintext P.
    - **Key**: 0x11111111111111111111111111111111
    - **IV**: 0x000000000000000000000000
    - **Ciphertext**: C9C5C1F5B599289A70761E60D0A4EA69DB41ABC40FE7E1EF38302F7009C2
    - **Plaintext**: This is a secret test message.
3. Decrypting the ciphertext C'' with key K and initialisation vector V' to recover the plaintext P.
    - **Key**: 0x00000000000000000000000000000000
    - **IV**: 0x111111111111111111111111
    - **Ciphertext**: 924BAD0C01B649365FA80F8FF5611BB81EEE0C6006D5DA7E7B91AB63CC76
    - **Plaintext**: This is a secret test message.

Based on the above result, each case confirmed that when the correct IV and key are used, the decryption process should reconstruct the original plaintext. This validates the correct application of the keystream regeneration and XOR mechanism.

C. Scenario 3 (Decryption with Wrong Input):

This scenario evaluates the cipher to generate "incorrect" output due to a mismatch in the key or IV. Since in a stream cipher, even a 1-bit change alters the whole keystream, resulting in incorrect output.

1. Decrypting the ciphertext C with a (wrong) key K' and initialisation vector V, where $K \neq K'$.
    - **Key**: 0x11111111111111111111111111111111
    - **IV**: 0x000000000000000000000000
    - **Ciphertext**: 63E8825E421DDF0C8B946209E692775F32A5AB1DF10CF0762697E1E72A5A
    - **Plaintext**: þE*Ø×í„¶§ÂUDøBÉeªŠË|ümÔ¯ðF¶
2. Decrypting the ciphertext C with a key K and a (wrong) initialisation vector V', where $V \neq V'$.
    - **Key**: 0x00000000000000000000000000000000
    - **IV**: 0x111111111111111111111111
    - **Ciphertext**: 63E8825E421DDF0C8B946209E692775F32A5AB1DF10CF0762697E1E72A5A
    - **Plaintext**: ¥ËF!cÂåµãp "?Âƒ ùGm.u+ãƒ
3. Decrypting the ciphertext C with a (wrong) key K' and a (wrong) initialisation vector V', where $K \neq K'$ and $V \neq V'$.
    - **Key**: 0x00000000000000000000000000000000
    - **IV**: 0x111111111111111111111111
    - **Ciphertext**: 63E8825E421DDF0C8B946209E692775F32A5AB1DF10CF0762697E1E72A5A
    - **Plaintext**: ¥ËF!cÂåµãp "?Âƒ ùGm.u+ãƒ

Based on the plaintext of each case, these tests confirm that the cipher behaves securely against mismatch IV/key and demonstrate resistance toward brute force attack or blind guess attacks on its decryption.

D. Scenario 4: Avalanche Effect Test

This scenario is designed to test the avalanche effect of the Grain128v2, where a 1-byte input difference, either in the key, IV, or plaintext, should result in a completely new ciphertext.

1. One bit key differences

	Case 1	Case 2
Key	0xFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFF	0xFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFE
IV	0xABCDEFABCDEFABCDEFABCDEF	
Plaintext	This tests the cipher's avalanche property.	
Ciphertext	99080F7C3032D6374716404EFB33B3FB77A2ABD4132A23B1F212107560766E6C66DCDF8D99F8D639234273	920D037C3A8A831452744D811FF4AF97C95184EA55AB949B1D487BECB59CB3055CC91944A96DBBAD394AC3

2. One-bit IV differences

	Case 1	Case 2
Key	0xFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFF	

IV	0xABCDEFABCDEFABCDEFABCDEF	0xABCDEFABCDEFABCDEFABCDEE
Plaintext	This tests the cipher's avalanche property.	
Ciphertext	99080F7C3032D6374716404EFB33B3FB77A2ABD4132A23B1F212107560766E6C66DCDF8D99F8D639234273	F3E2FD41DE4DB7FD67B5DCEFF21A06F583BAC9072F3948FABB65C41308638CA8D1A4A6BEA35A3FEE63BF5A

3. One-bit Plaintext Differences

	Case 1	Case 2
Key	0xFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFF	
IV	0xABCDEFABCDEFABCDEFABCDEF	
Plaintext	This tests the cipher's avalanche property.	This tests the cipher's avalanche propertY.
Ciphertext	99080F7C3032D6374716404EFB33B3FB77A2ABD4132A23B1F212107560766E6C66DCDF8D99F8D639234273	F3E2FD41DE4DB7FD67B5DCEFF21A06F583BAC9072F3948FABB65C41308638CA8D1A4A6BEA35A3FEE63BF5A

From the result, despite the input differing in 1 bit, the ciphertexts are completely different, which demonstrates the cipher's strong diffusion. This confirms that the graind128AEADv2 succeeds in providing robustness in its key handling variation.

E. Scenario 5: Input Edge Cases

Input edge case scenarios are tested to reveal the ability of the grain128AEADv2 to encrypt various cases of plaintext.

1. Empty Plaintext

Key	0x123456789012345678901234567890 12
IV	0x123456789012345678901234
Plaintext	**empty plaintext*
Ciphertext	**empty ciphertext*

The result is that no ciphertext is produced. In the user input handling, encrypting an empty plaintext is unobtainable, as it prevents and notify the user from entering a valid plaintext.

2. Single Character Plaintext

Key	0x123456789012345678901234567890 12
IV	0x123456789012345678901234
Plaintext	A

Ciphertext	F0

3. Very Long Plaintext

Key	0x123456789012345678901234567890 12
IV	0x123456789012345678901234
Plaintext	Lorem Ipsum Lorem Ipsum Lorem Ipsum Lorem Ipsum Lorem Ipsum Lorem Ipsum Lorem Ipsum Lorem Ipsum Lorem Ipsum Lorem Ipsum Lorem Ipsum Lorem Ipsum **continue till 10000x (119999 characters)*
Ciphertext	FDB316CF3C60DA4B770FAE1301AFA4C042299039C 656F84690BEC032534BA766420027FCA81531DD749 E7F9BD85312D2B5B0A5D3A0890DAB76DC2E8DA9 26704F421712427ECC49CBA56450562B980045E6832 A619E280FE405DF4569F5F9B83547910D2C09B79044 F2B17802AB6452470C7A9F7B77CDCF87716D229F08 23F197C6DDB7D1C76B36DC700E43DB278AC4F5C3 **total of 239998 characters*

Based on the ciphertext and usage of repeat detection, there is no repetition of characters longer than 16 bytes that was found. This shows the ciphertext does not exhibit any detectable pattern, which shows the characteristic of a secure cipher.

The result of the scenario above shows that the grain128AEADv2 can receive various types of input varying in size.

F. Scenario 6: Encryption/Decryption with wrong input (Key/IV) length

This test is conducted to review the grain128AEAD action in reacting to incorrect key or IV length input. A secure implementation should reject any inputs that do not meet the specific requirement, which is a 128-bit key and a 96-bit key.

1. Short Key and IV

Key	0x1234567890123456789012345678 90 *(1 byte shorter)*
IV	0x12345678901234567890 *(2 byte shorter)*
Plaintext	"Test Message"

Error Handling:

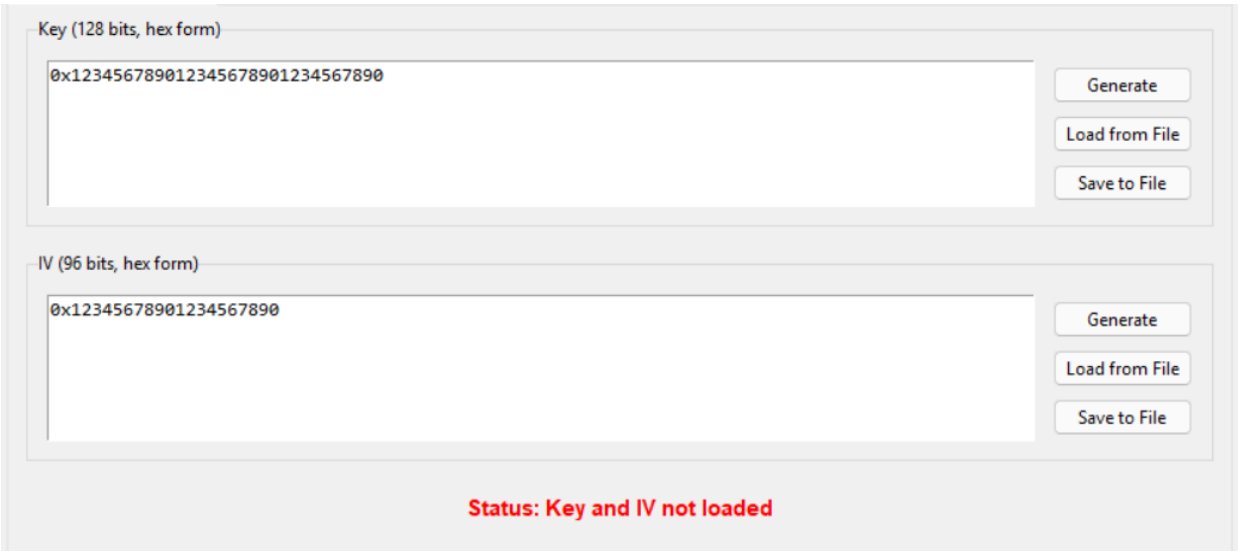


Image 2.1: Error Handling in the setup key and IV tab.

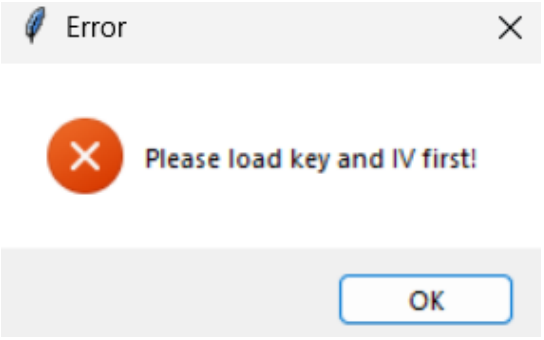


Image 2.2: Notice from encryption tab do to invalid key or IV input.

Based on the result, due to short keys and nonce, the user error handling shows that the key and IV are not loaded (see **Image 2.1**). When the user goes to the encrypt tab, it shows that the key and IV need to be loaded first (see **Image 2.2)**.

2. Long IV and Key

Key	0x12345678901234567890123456789012345678 90
IV	0x123456789012345678901234567890
Plaintext	“Test Message”

Error Handling:

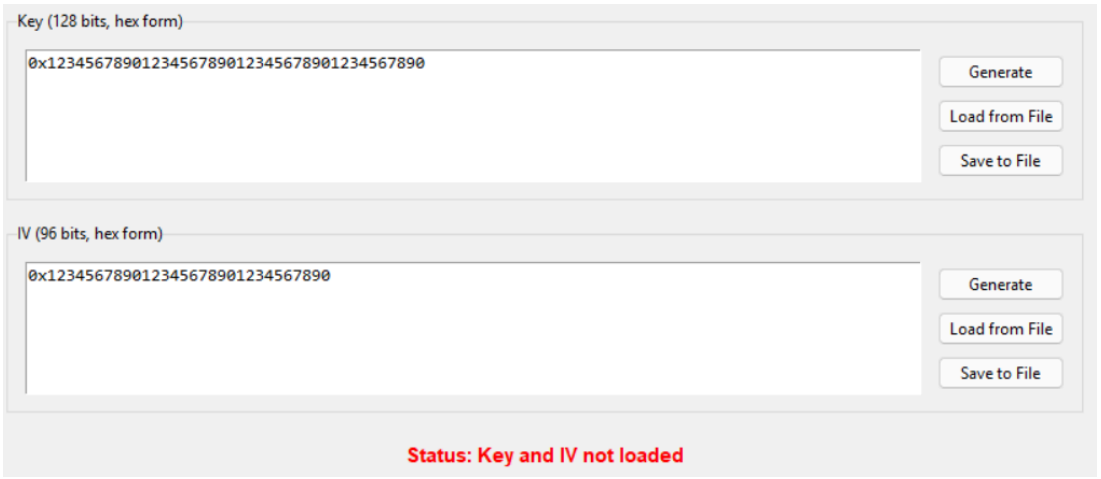


Image 2.3: Error Handling in the setup key and IV tab.

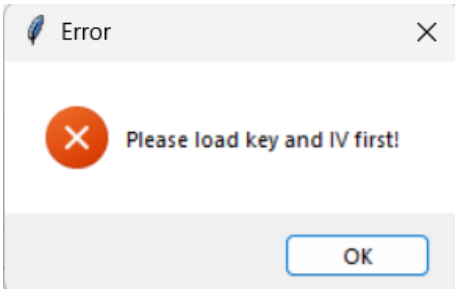


Image 2.4: Notice from encryption tab do to invalid key or IV input.

Based on the result, even if the keys and IV are more than enough, it is still stated as not loaded, and it needs proper input for it to process the encryption. As seen from both results, the program correctly validates the input length and demonstrates great user handling, and prevents errors in the algorithm.

G. Scenario 7: Determinism Test

To verify the consistency of the algorithm, the same key, IV, and plaintext are run multiple times. If it is deterministic, the output/ciphertext should be the same for all the runs.

1. Repeat Encryption

	Case 1	Case 2	Case 3
Key	0xFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFF		
IV	0x102030405060708090A0B0C0		
Plaintext	This tests the cipher's deterministic property.		
Ciphertext	1AE862D33F703F222A117EE22BFC7956C1AF3A2C3	1AE862D33F703F222A117EE22BFC7956C1AF3A2C3	1AE862D33F703F222A117EE22BFC7956C1AF3A2C3

	E1A19F4FE95B95 7AE5DDD6C7AD 9EF9C36714EEAF 938C4B2F382B1	E1A19F4FE95B95 7AE5DDD6C7AD 9EF9C36714EEAF 938C4B2F382B1	E1A19F4FE95B95 7AE5DDD6C7AD 9EF9C36714EEAF 938C4B2F382B1

2. Repeat Decryption

	Case 1	Case 2	Case 3
Key	0xFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFF		
IV	0x10203040506070809 0A0B0C0		
Ciphertext	1AE862D33F703F222A117EE22BFC7956C1AF3A2C3E1A1 9F4FE95B957AE5DDD6C7AD9EF9C36714EEAF938C4B2F 382B1		
Plaintext	This tests the cipher's deterministic property.	This tests the cipher's deterministic property.	This tests the cipher's deterministic property.

Based on the two tests, it can be seen that the ciphertext in repeating encryption and the plaintext in repeating decryption give identical results, which ensures the cipher's deterministic property.

3. Construction Analysis

1. Justification for 512 Initialization Rounds in Grain-128AEADv2

The Grain-128AEADv2 cipher requires 512 initialization rounds before outputting any keystream. The process itself is crucial in ensuring the cipher's confidentiality and

security, which is directly related to the internal state of the cipher. The 512 rounds are divided into three phases, with each phase playing a specific role in mixing the internal state.

a. Phase 1 (round 0 - 319): nonlinear mixing with feedback

    It uses the y_t or nonlinear feedback to mix the key and IV that are in the LSFR and NSFR. Additionally, as the nonlinear feedback relies on both LFSR and NSFR, it causes mutual mixing by spreading nonlinearity in both registers, making it hard to predict and full dependency on the key and IV. Reducing the rounds could result in incomplete diffusion, which means certain bits in the internal states do not depend on certain key/IV bits.

b. Phase 2 (round 329 - 283): key introduction

    In this 64-round, the output y_t is used again, but there is a key re-injection into the internal state. This strengthens the relationship between the internal state and the presence of the key. It reuses the first 64 key bits to achieve sufficient key dependence. This sets the foundation of reliability towards the key, further preventing exploitation of known IV attacks as the IV is public.

    Phase 2 rounds' contribution to security is crucial to make the internal state key-dependent, prevent partial reversibility, and prevent IV reuse leakage.

c. Phase 3 (round 384 - 511): warming up before keystream outputting

    In the last 128 rounds, the LSFR and NSFR are clocked without using y_t. The purpose of the last phase rounds is to “destroy” the internal structure from phase 2, which is heavily influenced by the key structure. Outputting it directly would expose the relationship between the key and the states. The last round is done to ensure that the keystream generated appears random and has less correlation with the key and IV.

Why Reducing the Initialization Rounds is Insecure

Reducing the number of initialization rounds from 512 to 12 rounds is risky for the security and confidentiality of the cipher. Even if the phases are retained, the number of rounds is crucial; it is not arbitrary and is carefully designed to ensure a significant number of rounds without compromising throughput.

Reducing it to 12 rounds would make the internal state retain more than half of its original state, as it did not mix the initial key and IV thoroughly. Additionally, it won't have strong binding with the key, and a low entropy state, which is vulnerable to known-IV attacks or distinguishing attacks. Hence, the avalanche effect is little to non-existent, and the 12-round state might mimic closer to a linear system, making correlation attack practical.

Reference Example

Modification was made to the initialization part of the code. It will run for 12 rounds, keeping the phases, with phase 1: 7 rounds, phase 2: 2 rounds, and phase 3: 3 rounds. The number of rounds is chosen to replicate the 512-round of the initial initialization part. The result will focus on the lack of rounds in the initialization. Below is the comparison between the initial state for 12-round initialization and the full 512-round initialization.

<table>
<tr><th colspan="2">CASE 1</th></tr>
<tr><td>12-Round</td><td>512-Round</td></tr>
<tr><td colspan="2">Key: 0x00000000000000000000000000000000</td></tr>
<tr><td colspan="2">IV: 0x000000000000000000000000</td></tr>
<tr><td>[After Loading – 12 Rounds Mode]
NFSR: 00000000000000000000000000000000
LFSR: 0000000000000000000000FFFFFF7F</td><td>[After Loading]
NFSR: 00000000000000000000000000000000
LFSR: 0000000000000000000000FFFFFF7F</td></tr>
<tr><td>[After 12 Rounds Initialization – Insecu
NFSR: 0000000000000000000000000080FF
LFSR: 00000000000000000000F0FFFFFF7700</td><td>[After Initialization]
NFSR: 81F7E0C655D035823310C278438DBC20
LFSR: 8F395A9421B0963364E2ED30679C8EE1</td></tr>
</table>

CASE 2	
12-Round	512-Round
Key: 0x000102030405060708090a0b0c0d0e0f	
IV: 0x000102030405060708090a0b	
[After Loading – 12 Rounds Mode] NFSR: 000102030405060708090A0B0C0D0E0F LFSR: 000102030405060708090A0BFFFFFF7F	[After Loading] NFSR: 000102030405060708090A0B0C0D0E0F LFSR: 000102030405060708090A0BFFFFFF7F
[After 12 Rounds Initialization – Insec NFSR: 2030405060708090A0B0C0D0E0F0B087 LFSR: 2030405060708090A0B0F0FFFFFFD748	[After Initialization] NFSR: B3C2E1B1EEC1F08C2D6EAE957F6AF9D0 LFSR: 0E1F950D45E05087C4CD63FD00EAB310

CASE 3	
12-Round	512-Round
Key: 0x00000000000000000000000000000000	
IV: 0x111111111111111111111111	
[After Loading – 12 Rounds Mode] NFSR: 00000000000000000000000000000000 LFSR: 111111111111111111111111FFFFFF7F	[After Loading] NFSR: 00000000000000000000000000000000 LFSR: 111111111111111111111111FFFFFF7F
[After 12 Rounds Initialization – Insec NFSR: 00000000000000000000000000090EE LFSR: 111111111111111111F1FFFFFFC7BB	[After Initialization] NFSR: 3EE1F9CF14E3FD0D2D8762BBFE024811 LFSR: 9933FB6E66CE17109BB2121D29AB420A

From all tested cases, a recurring pattern is observed in the 12-round initialization, especially within the NSFR, where minimal transformation occurred, with only 2 bytes shifted and 2 new bytes introduced. Specifically, in the LSFR from visual inspection, the following pattern can be identified:

1. The 1- 20 hex character of the LSFR remains unaffected by any changes in the NSFR, indicating a lack of effective mixing between the two registers.
2. The 21st hex character consistently appears as F.
3. The 22nd hex character mirrors the 23rd hex character of the LFSR. (See **image 3.1**).
4. Hex characters from positions 23 to 28 are consistently filled with F.

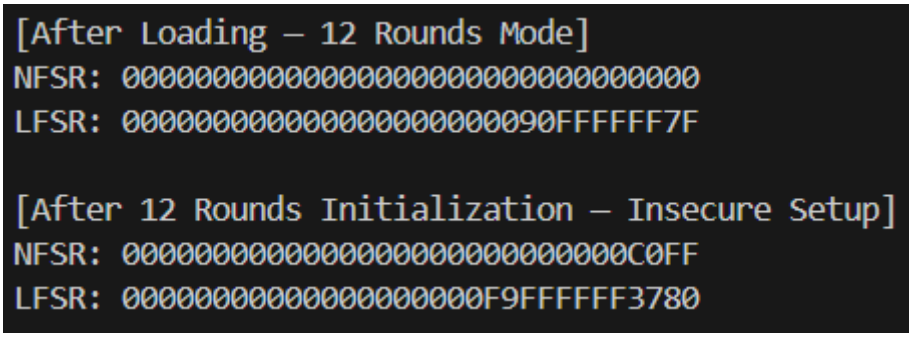


Image 3.1: The number '9' is in the 23rd hex character

After a few more test cases, the result of these 2 cases confidently defines the shift, the slight transformation of the LSFR, which exposed the pattern and therefore vulnerable to attacks.

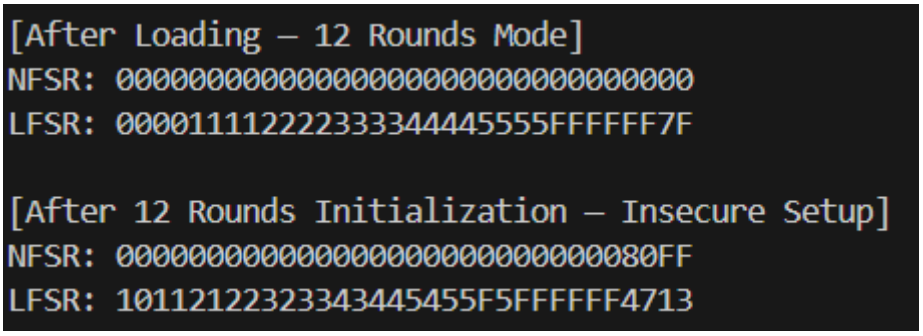


Image 3.2: Extremely weak mixing of LSFR

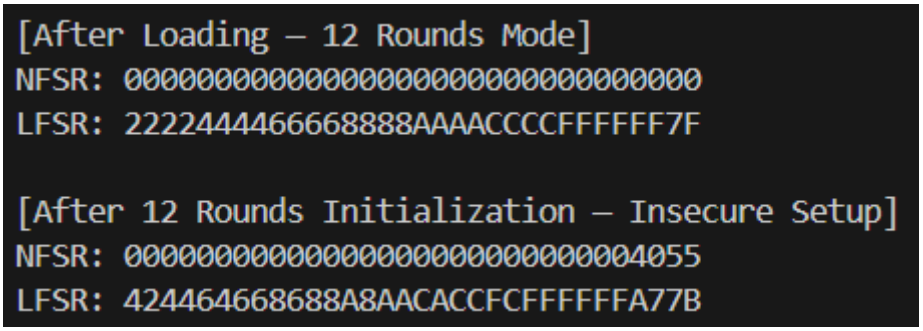


Image 3.3: Extremely weak mixing of LSFR

The LSFR internal state is shown below based on **Image 3.2**

[Before Initialization] LFSR:

00001111222233344445555FFFFFF7F

[After 12 Rounds] LFSR:

1011212232343445455F5FFFFFF4713

After analyzing the LSFR, a pattern is shown:

- 00001111 became 10112122
- 22223333 became 21223233

This shows the 12 rounds are sufficient to provide a non-linear mixing, which risks the overall security and confidentiality of the internal state, key, and IV. With the IV being public knowledge, the LSFR internal state could be computed and risk the security of the whole plaintext. The internal state is not sufficiently scrambled, and the intended avalanche effect was not created. Therefore, reducing the 512-rounds to 12 rounds to speed up the cipher is highly flawed and would weaken the cipher's security as a whole, rendering the grain128AEADv2 unusable as it violates the principle of its intended design.

2. Rationale for Reintroducing the Key During Clocks 320 to 383

One of the key differences between grain128AEAD with grain128AEADv2 is the key reintroduction during the initialization of rounds 320 to 383. The extra 64 rounds were added to mitigate the risk of the attacker's ability to reverse the internal shift to extract the key. As in earlier versions, under the assumption that the attacker has complete knowledge of the

cipher’s internal state, it is theoretically possible for the attacker to invert the shifting operation and exploit the key.

To counter the inversion of the shift operation, the key is reintroduced to the LSFR and NSFR internal state. This reintroduction breaks direct reversibility between the internal state and the key. Ensuring that knowing the whole internal state is not sufficient to deduce the secret key. Additionally, adding the key into the state after 320 rounds makes nonlinear dependencies on the states. This approach increases the computational complexity of key recovery. Hence, under the assumption that the attacker itself has full access to the cipher’s internal state after the initialization, the attacker won’t be able to recover the key. As a result, key reintroduction is beneficial in increasing the confidentiality and security of the frain128AEADv2 stream cipher.

Appendix A: Source Code

1. main.py

```
import hashlib # for SHA256

import random #for key and iv random generation

import os



#-----------------------------------------------------

# KEY ENCRYPTION AND DECRYPTION UTILITY FUNCTIONS

#-----------------------------------------------------



# SHA256 Utils

def sha256_hash(data):

    """SHA256 hash function"""

    if isinstance(data, str):

        data = data.encode('utf-8')

    return hashlib.sha256(data).digest()



def hash_password_to_key(password):

    """Hash password using SHA256"""

    return sha256_hash(password)



def xor_encrypt_key(key_bits, hash_bytes):

    """XOR encrypt key with hash (first 16 bytes)"""
```

```
    key_bytes = bits_to_bytes(key_bits[:128])
    encrypted = bytearray()
    for i in range(16):
        encrypted.append(key_bytes[i] ^ hash_bytes[i])
    return bytes(encrypted)


def save_encrypted_key(filename, enc_key):
    """Save encrypted key to file"""
    with open(filename, 'w') as f:
        for byte in enc_key:
            f.write(f"{byte:02X} ")


def load_encrypted_key(filename):
    """Load encrypted key from file"""
    with open(filename, 'r') as f:
        hex_bytes = f.read().strip().split()
        return bytes(int(hex_byte, 16) for hex_byte in hex_bytes)


def encrypt_key_with_password(original_key_bits, password,
output_file):
    """Encrypt key with password and save to file"""
    hash_bytes = hash_password_to_key(password)
    enc_key = xor_encrypt_key(original_key_bits, hash_bytes)
    save_encrypted_key(output_file, enc_key)
```

```
def decrypt_key_with_password(input_file, password):
    """Decrypt key with password from file"""
    hash_bytes = hash_password_to_key(password)
    enc_key = load_encrypted_key(input_file)

    decrypted_bytes = bytearray()
    for i in range(16):
        decrypted_bytes.append(enc_key[i] ^ hash_bytes[i])

    return bytes_to_bits(bytes(decrypted_bytes))

def hex_string_to_bits(hex_string, bit_length):
    """Convert MSB-first hex string to LSB-first bits (for
internal use)"""
    hex_string = hex_string.lower().strip()
    if hex_string.startswith("0x"):
        hex_string = hex_string[2:]

    # Keep only hex characters
    hex_string = ''.join(c for c in hex_string if c in
'0123456789abcdef')
    expected_hex_len = (bit_length + 3) // 4
    if len(hex_string) < expected_hex_len:
```

```
        raise ValueError(f"Hex string too short. Expected at least
{expected_hex_len} hex characters.")



    bits = []

    for i in range(0, len(hex_string), 2):  # Process each byte (2
hex chars)

        if i + 1 >= len(hex_string): break

        byte_val = int(hex_string[i:i+2], 16)

        # Convert MSB-first byte to LSB-first bits

        for j in range(8):

            bits.append((byte_val >> j) & 1)

    return bits[:bit_length]



def bits_to_bytes(bits):

    """Convert bits array to bytes (LSB-first per byte)"""

    bytes_array = bytearray()

    for i in range(0, len(bits), 8):

        byte = 0

        for j in range(8):

            if i + j < len(bits):

                byte |= (bits[i + j] << j)

        bytes_array.append(byte)

    return bytes(bytes_array)
```

```
def bytes_to_bits(byte_data):
    """Convert bytes to bits array (LSB-first per byte)"""
    bits = []
    for byte in byte_data:
        for i in range(8):
            bits.append((byte >> i) & 1)
    return bits


def bits_to_hex_string(bits):
    """Convert LSB-first bits to MSB-first hex string for
display"""
    hex_chars = []
    for i in range(0, len(bits), 8):
        byte = 0
        for j in range(8):
            if i + j < len(bits):
                byte |= (bits[i + j] << j)
        hex_chars.append(f"{byte:02X}")
    return ''.join(hex_chars)



def load_hex_bits_from_file(filename, expected_bits):
    """Load hex string from file and convert to bits"""
    try:
```

```
        with open(filename, 'r') as f:
            hex_str = f.read().strip()
    except FileNotFoundError:
        print(f"Error: File '{filename}' not found.")
        return None
    except Exception as e:
        print(f"Error reading file: {e}")
        return None

    if hex_str.lower().startswith("0x"):
        hex_str = hex_str[2:]

    # Filter valid hex characters
    hex_str = ''.join(c for c in hex_str if c.isdigit() or
c.lower() in 'abcdef')

    if len(hex_str) % 2 != 0:
        hex_str = hex_str[:-1]

    if not hex_str:
        print("Error: Empty file.")
        return None

    bits = []
```

```
    bit_index = 0
    for i in range(0, len(hex_str), 2):
        if bit_index >= expected_bits:
            break
        byte_val = int(hex_str[i:i+2], 16)
        for b in range(8):
            if bit_index < expected_bits:
                bits.append((byte_val >> b) & 1)
                bit_index += 1

    if len(bits) < expected_bits:
        raise ValueError(f"Not enough bits in hex input
({len(bits)} < {expected_bits} expected)")

    return bits[:expected_bits]

def read_plaintext(filename):
    """Read plaintext from file and convert to bits (LSB-first)"""
    with open(filename, 'r', encoding='utf-8') as f:
        text = f.read()

    bits = []
    for char in text:
        char_code = ord(char)
```

```
        # FIXED: Use LSB-first bit ordering
        for i in range(8):
            bits.append((char_code >> i) & 1)

    return bits

def read_ciphertext(filename):
    """Read ciphertext from hex file and convert to bits
(LSB-first)"""
    with open(filename, 'r') as f:
        hex_content = f.read().replace(' ', '').replace('\n', '')

    bits = []
    for i in range(0, len(hex_content), 2):
        if i + 1 < len(hex_content):
            byte_val = int(hex_content[i:i+2], 16)
            # FIXED: Use LSB-first bit ordering
            for j in range(8):
                bits.append((byte_val >> j) & 1)

    return bits

def generate_random_bits_to_file(filename, bit_len):
```

```
    """Generate random hex bits and save to file, for command line
user interface"""

    hex_len = bit_len // 4

    random_hex = ''.join(f"{random.randint(0, 15):X}" for _ in
range(hex_len))



    with open(filename, 'w') as f:

        f.write(random_hex)



def text_to_bits(text):

    """Convert text string to bits (LSB-first)"""

    bits = []

    for char in text:

        char_code = ord(char)

        # FIXED: Use LSB-first bit ordering

        for i in range(8):

            bits.append((char_code >> i) & 1)

    return bits



#-----------------------------------------------

# Grain-128 AEAD Implementation

#-----------------------------------------------

STATE_SIZE = 128
```

```
class Grain128AEAD:

    def __init__(self):

        self.LFSR = [0] * STATE_SIZE #128 bit LSFR

        self.NFSR = [0] * STATE_SIZE #128 bit NSFR



        #storage for key and IV

        self.key = [0] * 128

        self.iv = [0] * 128



    def L(self, S):

        """LFSR update function"""

        return S[0] ^ S[7] ^ S[38] ^ S[70] ^ S[81] ^ S[96]



    def F(self, B):

        """NFSR update function"""

        # update function = s[127] = s[0] + s[7] + s[38] + s[70] +
s[81] + s[96]

        return (B[0] ^ B[26] ^ B[56] ^ B[91] ^ B[96] ^

                (B[3] & B[67]) ^ (B[11] & B[13]) ^

                (B[17] & B[18]) ^ (B[27] & B[59]) ^

                (B[40] & B[48]) ^ (B[61] & B[65]) ^ (B[68] &
B[84]) ^

                (B[22] & B[24] & B[25]) ^ (B[70] & B[78] & B[82])
^
```

```
                (B[88] & B[92] & B[93] & B[95])) # returns new bit
for position 127



    def h(self, S, B):

        """Nonlinear Boolean function h(x)"""

        # S (list): LFSR state, B (list): NFSR state

        # 2 from NSFR, 7 from LSFR

        x0, x1, x2, x3, x4, x5, x6, x7, x8 = B[12], S[8], S[13],
S[20], B[95], S[42], S[60], S[79], S[94]

        return (x0 & x1) ^ (x2 & x3) ^ (x4 & x5) ^ (x6 & x7) ^ (x0
& x4 & x8)



    def y_t(self, S, B):

        """Pre-output function -> generates the output bit"""

        #  y_t = h(S,B) + S[93] + sum(B[j] for j in A)

        # where A = {2, 15, 36, 45, 64, 73, 89}

        return (self.h(S, B) ^ S[93] ^

                B[2] ^ B[15] ^ B[36] ^ B[45] ^ B[64] ^ B[73] ^
B[89])



    def shift_left(self, reg):

        """Shift register left"""

        for i in range(STATE_SIZE - 1):

            reg[i] = reg[i + 1]
```

```
        reg[STATE_SIZE - 1] = 0    # Position 127 receives a new
bit.


    def initialize(self, key_bits, iv_bits):
        """Initialize the cipher with key and IV

        Initialize the cipher with key and IV following the
3-phase process.

        The initialization process has three phases:
        1. Phase 1 (t=0-319): Clock with pre-output feedback to
both registers
        2. Phase 2 (t=320-383): Clock with key feedback and
pre-output feedback
        3. Phase 3 (t=384-511): Clock without any feedback
(keystream generation setup)

        Args:
            key_bits (list): 128-bit key as bit array
            iv_bits (list): 96-bit IV as bit array
        """

        self.key = key_bits[:128] # store key and IIV
        self.NFSR = key_bits[:128].copy() # load NSFR with key
bits
```

```
        self.iv = iv_bits[:96] + [0] * 32 # load IV bits, with 0
extended

        self.LFSR = iv_bits[:96].copy()



        self.LFSR.extend([1] * 31 + [0]) # last 32 bits = 31 ones
and 1 zero



        # Print after loading (USE FOR TESTING/ DATA VALIDATION)

        print("\n[After Loading]")

        print("NFSR:", bits_to_hex_string(self.NFSR))

        print("LFSR:", bits_to_hex_string(self.LFSR))



        # Phase 1: 0-319

        # both received pre-output feedback

        for t in range(320):

            y = self.y_t(self.LFSR, self.NFSR)



            # LFSR update: s[127] = L(S) + y_t

            new_L = self.L(self.LFSR) ^ y



            # NFSR update: b[127] = s[0] + F(B) + y_t

            new_N = self.LFSR[0] ^ self.F(self.NFSR) ^ y



            # update both registers
```

```
            self.shift_left(self.LFSR)

            self.shift_left(self.NFSR)



            #insert new bits

            self.LFSR[127] = new_L

            self.NFSR[127] = new_N



        # Phase 2: 320-383

        # key bits with pre-output is fed into register

        for t in range(320, 384):

            y = self.y_t(self.LFSR, self.NFSR)



            # LFSR = s[127] = L(S) + y_t + k[t-256]

            new_L = self.L(self.LFSR) ^ y ^ self.key[t - 256]



            # NFSR gets different key bit and pre-output:

            # b[127] = s[0] + F(B) + y_t + k[t-320]

            new_N = self.LFSR[0] ^ self.F(self.NFSR) ^ y ^
self.key[t - 320]



            # update both registers

            self.shift_left(self.LFSR)

            self.shift_left(self.NFSR)
```

```
            # insert new bits
            self.LFSR[127] = new_L
            self.NFSR[127] = new_N

        # Phase 3: 384-511 (no feedback)
        for t in range(384, 512):
            # register update with no feedback
            new_L = self.L(self.LFSR)
            new_N = self.LFSR[0] ^ self.F(self.NFSR)

            # update both register
            self.shift_left(self.LFSR)
            self.shift_left(self.NFSR)

            # insert new bits
            self.LFSR[127] = new_L
            self.NFSR[127] = new_N

        # Print after initialization (USE FOR TESTING/ DATA
VALIDATION)
        print("\n[After Initialization]")
        print("NFSR:", bits_to_hex_string(self.NFSR))
        print("LFSR:", bits_to_hex_string(self.LFSR))
```

```
    def update_registers(self):
        """Update the LFSR and NFSR registers by shifting and
inserting new values."""
        new_L = self.L(self.LFSR)
        new_N = self.LFSR[0] ^ self.F(self.NFSR)  # S[0] ⊕ F(NFSR)

        self.shift_left(self.LFSR)
        self.shift_left(self.NFSR)
        self.LFSR[127] = new_L
        self.NFSR[127] = new_N

    def generate_keystream_bit(self):
        """Generate one keystream bit, takes even bit only"""
        # Generate keystream bit (even bit y512+2i)
        keystream_bit = self.y_t(self.LFSR, self.NFSR)

        # Update registers
        self.update_registers()

        # Generate and discard the authentication bit (odd bit
y512+2i+1)
        auth_bit = self.y_t(self.LFSR, self.NFSR)  # Generate but
don't use
```

```
        # Update registers again
        self.update_registers()

        return keystream_bit

    def encrypt(self, plaintext_bits, output_file="cipher.enc"):
        """Encrypt plaintext bits (c_i = m_i ⊕ z_i)

         plaintext_bits (list): Plaintext as bit array
         output_file (str): Output filename for ciphertext"""
        # encrypt bit by bit
        cipher_bits = []
        for i in range(len(plaintext_bits)):
            keystream_bit = self.generate_keystream_bit()
            cipher_bits.append(plaintext_bits[i] ^ keystream_bit)

        # Convert LSB-first bits to hex output/file
        with open(output_file, "w") as f:
            for i in range(0, len(cipher_bits), 8):
                byte = 0
                # reconstruct byte from LSB-first bits
                for j in range(8):
                    if i + j < len(cipher_bits):
```

```
                        byte |= (cipher_bits[i + j] << j)  #
LSB-first
                f.write(f"{byte:02X}")


        return cipher_bits # returns encrypted bits


    def decrypt(self, ciphertext_bits, output_file):
        """Decrypt ciphertext bits (  m_i = c_i ⊕ z_i )


         ciphertext_bits (list): Ciphertext as bit array
         output_file (str): Output filename for plaintext"""
        # Decrypt bit by bit
        plaintext_bits = []
        for i in range(len(ciphertext_bits)):
            keystream_bit = self.generate_keystream_bit()
            plaintext_bits.append(ciphertext_bits[i] ^
keystream_bit)


        # Convert LSB-first bits to proper characters
reconstruction
        with open(output_file, "w", encoding='utf-8') as f:
            for i in range(0, len(plaintext_bits), 8):
                byte = 0
                for j in range(8):
```

```
                    if i + j < len(plaintext_bits):
                        byte |= (plaintext_bits[i + j] << j)  # 
LSB-first
                if byte > 0: #only write printable character
                    f.write(chr(byte))

        return plaintext_bits


#-----------------------------------------------
# MAIN PROGRAM AND COMMAND LINE USER INTERFACE
#-----------------------------------------------


# display main menu options
def menu():
    print("\n==== Grain-128AEADv2 Encryption Menu ====")
    print("1. Setup key and IV")
    print("2. Encryption")
    print("3. Decryption")
    print("0. Exit")
    choice = input("Select an option: ")
    return choice


def main():
```

```
    """Main program loop providing user interface for the
Grain-128AEAD cipher.


    This function provides a menu-driven interface allowing users
to:
    1. Set up 128-bit keys and 96-bit IVs (manually, from file, or
randomly)
    2. Encrypt plaintext (manual input or from file) additionally
encrypt key
    3. Decrypt ciphertext with password-protected keys"""
    cipher = Grain128AEAD()
    key_loaded = False
    iv_loaded = False
    key_bits = []
    iv_bits = []


    while True:
        choice = menu()


        if choice == '1':
            print("\nKey and IV Setup Menu:")
            print("1. Enter manually")
            print("2. Load from file")
            print("3. Generate randomly")
            sub_choice = input("Select an option: ")
```

```
            if sub_choice == '1':
                key_input = input("Enter 128-bit hex key (32 hex
chars): ").strip().lower()
                iv_input = input("Enter 96-bit hex IV (24 hex
chars): ").strip().lower()

                 # Strip 0x prefix if present
                if key_input.lower().startswith("0x"):
                    key_input = key_input[2:]
                if iv_input.lower().startswith("0x"):
                    iv_input = iv_input[2:]

                 # Remove whitespace and ensure only hex chars
remain
                key_input = ''.join(c for c in key_input if c in
'0123456789abcdef')
                iv_input = ''.join(c for c in iv_input if c in
'0123456789abcdef')

                if len(key_input) != 32 or len(iv_input) != 24:
                    print("Invalid input length.")
                    continue

                try:
```

```
                    key_bits = hex_string_to_bits(key_input, 128)
                    iv_bits = hex_string_to_bits(iv_input, 96)
                    key_loaded = True
                    iv_loaded = True
                    print("Manual key and IV retrieved
successfully.")
                except ValueError as e:
                    print(f"Error: {e}")

            elif sub_choice == '2':
                key_file = input("Enter filename for 128-bit key:
")
                iv_file = input("Enter filename for 96-bit IV: ")

                key_bits = load_hex_bits_from_file(key_file, 128)
                iv_bits = load_hex_bits_from_file(iv_file, 96)

                if key_bits is None or iv_bits is None:
                    print("Error loading key or IV from file.")
                    key_loaded = False
                    iv_loaded = False
                else:
                    key_loaded = True
                    iv_loaded = True
```

```
                    print("Key and IV retrieved from file
successfully.")



            elif sub_choice == '3':

                print("Generating random 128-bit key and 96-bit
IV...")

                generate_random_bits_to_file("key.txt", 128)

                generate_random_bits_to_file("iv.txt", 96)



                key_bits = load_hex_bits_from_file("key.txt", 128)

                iv_bits = load_hex_bits_from_file("iv.txt", 96)



                if key_bits is None or iv_bits is None:

                    print("Failed to load generated key/IV.")

                    key_loaded = False

                    iv_loaded = False

                else:

                    key_loaded = True

                    iv_loaded = True

                    print("Random key and IV generated and loaded
successfully.")

                    print("Key saved to key.txt")

                    print("IV saved to iv.txt")
```

```
        elif choice == '2':
            if not key_loaded or not iv_loaded:
                print("Error: Key and IV not loaded. Please load
them first.")
                continue

            cipher.initialize(key_bits, iv_bits)

            print("\nSelect plaintext input method:")
            print("1. Input manually")
            print("2. Load from file")
            method = input("Select an option: ")

            if method == '1':
                input_text = input("Enter plaintext: ")
                plaintext_bits = text_to_bits(input_text)
                print(f"Manual plaintext input loaded:
{len(plaintext_bits)} bits")

            elif method == '2':
                filename = input("Enter filename (e.g.,
plaintext.txt): ")
                plaintext_bits = read_plaintext(filename)
```

```
                if not plaintext_bits:
                    continue

            else:
                print("Invalid selection.")
                continue

            cipher.encrypt(plaintext_bits)
            print("Plaintext encrypted using Grain-128AEADv2.")
            print("Ciphertext saved to cipher.enc (hex format).")

            # Encrypt key with password
            output_file = input("Enter filename to save encrypted
key (e.g., encrypted.key): ")
            password = input("Enter password to encrypt the key:
")

            encrypt_key_with_password(key_bits, password,
output_file)
            print("Key encrypted and saved successfully.")

        elif choice == '3':
            if not iv_loaded:
                print("Error: Please load the IV first.")
```

```
                continue

            # Decrypt key with password
            key_file = input("Enter filename to load encrypted key
from (e.g., encrypted.key): ")
            password = input("Enter password to decrypt the key:
")

            try:
                key_bits = decrypt_key_with_password(key_file,
password)
                key_loaded = True
            except Exception as e:
                print(f"Error decrypting key: {e}")
                continue

            cipher_file = input("Enter ciphertext file name (with
.enc extension): ")
            ciphertext_bits = read_ciphertext(cipher_file)

            if not ciphertext_bits:
                continue

            # Create output filename
            if cipher_file.endswith('.enc'):
```

```
                output_file = cipher_file[:-4] + '.dec'
            else:
                output_file = cipher_file + '.dec'

            cipher.initialize(key_bits, iv_bits)
            cipher.decrypt(ciphertext_bits, output_file)

            print("Ciphertext decrypted successfully.")
            print(f"Decrypted text saved to {output_file}")

        elif choice == '0':
            print("Exiting.")
            break

        else:
            print("Invalid option. Try again.")

if __name__ == "__main__":
    main()
```

2. **gui.py**

```
import main

import tkinter as tk

from tkinter import ttk, messagebox, filedialog

import hashlib

import random

import os

from main import (

    Grain128AEAD,

    encrypt_key_with_password,

    decrypt_key_with_password,

    hex_string_to_bits,

    bits_to_bytes,

    bits_to_hex_string,

    load_hex_bits_from_file,

    read_plaintext,

    read_ciphertext,

    text_to_bits

)




# GUI Application

class GrainGUI:
```

```
    def __init__(self, root):
        self.root = root
        self.root.title("Grain-128 AEAD v2")
        self.root.geometry("850x650")  # window size

        # Configure style for larger tabs
        self.style = ttk.Style()
        self.style.configure('Larger.TNotebook.Tab', padding=[20, 12],
font=('TkDefaultFont', 10))
        self.style.configure('Large.TButton', font=('TkDefaultFont',
10))
        self.style.configure('Large.TLabel', font=('TkDefaultFont',
10))
        self.style.configure('Large.TLabelframe.Label',
font=('TkDefaultFont', 10))

        self.cipher = Grain128AEAD()
        self.key_bits = []
        self.iv_bits = []
        self.key_loaded = False
        self.iv_loaded = False

        self.create_widgets()

    def create_widgets(self):
```

```
        # Create notebook for tabs with larger style
        self.notebook = ttk.Notebook(self.root,
style='Larger.TNotebook')
        self.notebook.pack(fill=tk.BOTH, expand=True, padx=12, pady=12)
# Increased padding


        # Setup Tab
        self.tab_setup = ttk.Frame(self.notebook)
        self.notebook.add(self.tab_setup, text="Setup Key & IV")
        self.create_setup_tab(self.tab_setup)


        # Encryption Tab
        self.tab_encrypt = ttk.Frame(self.notebook)
        self.notebook.add(self.tab_encrypt, text="Encryption")
        self.create_encrypt_tab(self.tab_encrypt)


        # Decryption Tab
        self.tab_decrypt = ttk.Frame(self.notebook)
        self.notebook.add(self.tab_decrypt, text="Decryption")
        self.create_decrypt_tab(self.tab_decrypt)


        # Enhanced navigation buttons
        self.go_encrypt = tk.Button(self.tab_setup, text="Go to Encrypt
Tab",
```

```
                                      command=lambda:
self.notebook.select(self.tab_encrypt),

                                      font=('TkDefaultFont', 10, 'bold'),

                                      padx=15, pady=8,

                                      bg='#4CAF50', fg='white',

                                      relief='raised', bd=2)

        self.go_decrypt = tk.Button(self.tab_setup, text="Go to Decrypt
Tab",

                                      command=lambda:
self.notebook.select(self.tab_decrypt),

                                      font=('TkDefaultFont', 10, 'bold'),

                                      padx=15, pady=8,

                                      bg='#2196F3', fg='white',

                                      relief='raised', bd=2)



        # Keep them hidden at first

        self.go_encrypt.pack_forget()

        self.go_decrypt.pack_forget()




    def create_setup_tab(self, parent):

        # Key Section

        key_frame = ttk.LabelFrame(parent, text="Key (128 bits, hex
form)", padding=12)  # Increased padding

        key_frame.pack(fill=tk.X, padx=12, pady=8)  # Increased padding
```

```
        self.key_entry = tk.Text(key_frame, height=3, width=50,
font=('Consolas', 10))  # Added font
        self.key_entry.pack(side=tk.LEFT, fill=tk.BOTH, expand=True)

        # Bind events for auto-loading key
        self.key_entry.bind('<KeyRelease>', self.on_key_change)
        self.key_entry.bind('<Button-1>', self.on_key_change)  # For
mouse paste
        self.key_entry.bind('<Control-v>', self.on_key_paste)

        key_btn_frame = ttk.Frame(key_frame)
        key_btn_frame.pack(side=tk.RIGHT, padx=(12, 0))

        # Enhanced buttons with larger size
        ttk.Button(key_btn_frame, text="Generate",
command=self.generate_key,
                   width=14).pack(pady=4)  # Increased width and padding
        ttk.Button(key_btn_frame, text="Load from File",
command=self.load_key_file,
                   width=14).pack(pady=4)
        ttk.Button(key_btn_frame, text="Save to File",
command=self.save_key_file,
                   width=14).pack(pady=4)
```

```
        # IV Section
        iv_frame = ttk.LabelFrame(parent, text="IV (96 bits, hex
form)", padding=12)  # Increased padding
        iv_frame.pack(fill=tk.X, padx=12, pady=8)  # Increased padding

        self.iv_entry = tk.Text(iv_frame, height=3, width=50,
font=('Consolas', 10))  # Added font
        self.iv_entry.pack(side=tk.LEFT, fill=tk.BOTH, expand=True)

        # Bind events for auto-loading IV
        self.iv_entry.bind('<KeyRelease>', self.on_iv_change)
        self.iv_entry.bind('<Button-1>', self.on_iv_change)  # For
mouse paste
        self.iv_entry.bind('<Control-v>', self.on_iv_paste)

        iv_btn_frame = ttk.Frame(iv_frame)
        iv_btn_frame.pack(side=tk.RIGHT, padx=(12, 0))

        # Enhanced buttons with larger size
        ttk.Button(iv_btn_frame, text="Generate",
command=self.generate_iv,
                   width=14).pack(pady=4)  # Increased width and padding
        ttk.Button(iv_btn_frame, text="Load from File",
command=self.load_iv_file,
                   width=14).pack(pady=4)
```

```
        ttk.Button(iv_btn_frame, text="Save to File",
command=self.save_iv_file,
                   width=14).pack(pady=4)

        # Status
        self.status_label = ttk.Label(parent, text="Status: Key and IV
not loaded",
                                      foreground="red",
font=('TkDefaultFont', 11, 'bold'))  # Enhanced font
        self.status_label.pack(pady=12)

    def on_key_change(self, event=None):
        """Auto-load key when text changes"""
        # Use after_idle to ensure the text change is processed first
        self.root.after_idle(self.auto_load_key)

    def on_key_paste(self, event=None):
        """Handle paste event for key"""
        # Use after to allow paste to complete first
        self.root.after(10, self.auto_load_key)

    def on_iv_change(self, event=None):
        """Auto-load IV when text changes"""
        # Use after_idle to ensure the text change is processed first
```

```
        self.root.after_idle(self.auto_load_iv)


    def on_iv_paste(self, event=None):
        """Handle paste event for IV"""
        # Use after to allow paste to complete first
        self.root.after(10, self.auto_load_iv)


    def auto_load_key(self):
        """Automatically load key from text entry"""
        try:
            key_text = self.key_entry.get(1.0, tk.END).strip()
            # Remove 0x prefix if present (case insensitive)
            if key_text.lower().startswith('0x'):
                key_text = key_text[2:]
            # Filter to only hex characters
            key_text = ''.join(c for c in key_text if c.isdigit() or
c.lower() in 'abcdef')


            if len(key_text) == 32:  # Valid key length
                self.key_bits = hex_string_to_bits(key_text, 128)
                self.key_loaded = True
                self.update_status()
            elif len(key_text) == 0:  # Empty field
                self.key_loaded = False
```

```
                self.update_status()
            # For invalid lengths, we don't show error messages to
avoid spam
            # but we also don't load the key
            elif self.key_loaded and len(key_text) != 32:
                self.key_loaded = False
                self.update_status()
        except Exception:
            # Silently handle errors during auto-loading
            if self.key_loaded:
                self.key_loaded = False
                self.update_status()

    def auto_load_iv(self):
        """Automatically load IV from text entry"""
        try:
            iv_text = self.iv_entry.get(1.0, tk.END).strip()
            # Remove 0x prefix if present (case insensitive)
            if iv_text.lower().startswith('0x'):
                iv_text = iv_text[2:]
            # Filter to only hex characters
            iv_text = ''.join(c for c in iv_text if c.isdigit() or
c.lower() in 'abcdef')
```

```
            if len(iv_text) == 24:  # Valid IV length
                self.iv_bits = hex_string_to_bits(iv_text, 96)
                self.iv_loaded = True
                self.update_status()
            elif len(iv_text) == 0:  # Empty field
                self.iv_loaded = False
                self.update_status()
            # For invalid lengths, we don't show error messages to
avoid spam
            # but we also don't load the IV
            elif self.iv_loaded and len(iv_text) != 24:
                self.iv_loaded = False
                self.update_status()
        except Exception:
            # Silently handle errors during auto-loading
            if self.iv_loaded:
                self.iv_loaded = False
                self.update_status()


    def create_encrypt_tab(self, parent):
        # Plaintext Section
        plain_frame = ttk.LabelFrame(parent, text="Plaintext",
padding=12)  # Increased padding
        plain_frame.pack(fill=tk.BOTH, expand=True, padx=12, pady=8)
```

```
        self.plaintext_entry = tk.Text(plain_frame, height=10,
font=('Consolas', 10))  # Added font
        self.plaintext_entry.pack(fill=tk.BOTH, expand=True)

        plain_btn_frame = ttk.Frame(plain_frame)
        plain_btn_frame.pack(fill=tk.X, pady=(12, 0))

        ttk.Button(plain_btn_frame, text="Load from File",
command=self.load_plaintext_file,
                   width=15).pack(side=tk.LEFT)  # Increased width
        ttk.Button(plain_btn_frame, text="Encrypt and Save",
command=self.encrypt_text,
                   width=15).pack(side=tk.RIGHT)  # Increased width

        # Ciphertext Display Section
        cipher_display_frame = ttk.LabelFrame(parent, text="Ciphertext
(hex form)", padding=12)
        cipher_display_frame.pack(fill=tk.BOTH, expand=True, padx=12,
pady=8)

        self.ciphertext_display = tk.Text(cipher_display_frame,
height=8, font=('Consolas', 10),
                                          state=tk.DISABLED,
wrap=tk.WORD)
        self.ciphertext_display.pack(fill=tk.BOTH, expand=True)
```

```
        # Encrypt Key Section
        key_encrypt_frame = ttk.LabelFrame(parent, text="Encrypt Key",
padding=12)  # Increased padding
        key_encrypt_frame.pack(fill=tk.X, padx=12, pady=8)

        ttk.Label(key_encrypt_frame, text="Filename:",
font=('TkDefaultFont', 10)).grid(
            row=0, column=0, sticky=tk.W, padx=(0, 8))  # Enhanced font
and padding
        self.encrypt_key_filename = ttk.Entry(key_encrypt_frame,
width=35, font=('TkDefaultFont', 10))  # Increased width and font
        self.encrypt_key_filename.grid(row=0, column=1, padx=(0, 12))

        ttk.Label(key_encrypt_frame, text="Password:",
font=('TkDefaultFont', 10)).grid(
            row=1, column=0, sticky=tk.W, padx=(0, 8), pady=(8, 0))
        self.encrypt_key_password = ttk.Entry(key_encrypt_frame,
show="*", width=35, font=('TkDefaultFont', 10))  # Increased width and
font
        self.encrypt_key_password.grid(row=1, column=1, padx=(0, 12),
pady=(8, 0))

        ttk.Button(key_encrypt_frame, text="Encrypt Key",
command=self.encrypt_key,
                   width=12).grid(row=0, column=2, rowspan=2, padx=(8,
0))  # Increased width
```

```
    def create_decrypt_tab(self, parent):
        # Decrypt Key Section
        key_decrypt_frame = ttk.LabelFrame(parent, text="Decrypt Key",
padding=12)  # Increased padding
        key_decrypt_frame.pack(fill=tk.X, padx=12, pady=8)

        ttk.Label(key_decrypt_frame, text="Filename:",
font=('TkDefaultFont', 10)).grid(
            row=0, column=0, sticky=tk.W, padx=(0, 8))  # Enhanced font
and padding
        self.decrypt_key_filename = ttk.Entry(key_decrypt_frame,
width=30, font=('TkDefaultFont', 10))  # Increased width and font
        self.decrypt_key_filename.grid(row=0, column=1, padx=(0, 12))
        ttk.Button(key_decrypt_frame, text="Browse",
command=self.browse_decrypt_key,
                   width=10).grid(row=0, column=2)  # Increased width

        ttk.Label(key_decrypt_frame, text="Password:",
font=('TkDefaultFont', 10)).grid(
            row=1, column=0, sticky=tk.W, padx=(0, 8), pady=(8, 0))
        self.decrypt_key_password = ttk.Entry(key_decrypt_frame,
show="*", width=30, font=('TkDefaultFont', 10))  # Increased width and
font
        self.decrypt_key_password.grid(row=1, column=1, padx=(0, 12),
pady=(8, 0))
```

```
        ttk.Button(key_decrypt_frame, text="Decrypt Key",
command=self.decrypt_key,

                   width=12).grid(row=1, column=2, padx=(8, 0), pady=(8,
0))  # Increased width



        # Ciphertext Section

        cipher_frame = ttk.LabelFrame(parent, text="Ciphertext (hex
form)", padding=12)  # Increased padding

        cipher_frame.pack(fill=tk.BOTH, expand=True, padx=12, pady=8)



        cipher_input_frame = ttk.Frame(cipher_frame)

        cipher_input_frame.pack(fill=tk.X, pady=(0, 12))



        ttk.Label(cipher_input_frame, text="Ciphertext File:",
font=('TkDefaultFont', 10)).pack(side=tk.LEFT)  # Enhanced font

        self.cipher_filename = ttk.Entry(cipher_input_frame, width=45,
font=('TkDefaultFont', 10))  # Increased width and font

        self.cipher_filename.pack(side=tk.LEFT, padx=(8, 8), fill=tk.X,
expand=True)

        ttk.Button(cipher_input_frame, text="Browse",
command=self.browse_cipher_file,

                   width=10).pack(side=tk.RIGHT)  # Increased width



        ttk.Button(cipher_frame, text="Decrypt",
command=self.decrypt_text,

                   width=15).pack(pady=8)  # Increased width and padding
```

```
        # Results
        self.result_text = tk.Text(cipher_frame, height=8,
font=('Consolas', 10))  # Added font
        self.result_text.pack(fill=tk.BOTH, expand=True, pady=(12, 0))

    def generate_key(self):
        try:
            hex_len = 128 // 4
            random_hex = ''.join(f"{random.randint(0, 15):X}" for _ in
range(hex_len))
            self.key_entry.delete(1.0, tk.END)
            self.key_entry.insert(1.0, random_hex)
            # Auto-loading will handle the rest
        except Exception as e:
            messagebox.showerror("Error", f"Failed to generate key:
{str(e)}")

    def generate_iv(self):
        try:
            hex_len = 96 // 4
            random_hex = ''.join(f"{random.randint(0, 15):X}" for _ in
range(hex_len))
            self.iv_entry.delete(1.0, tk.END)
            self.iv_entry.insert(1.0, random_hex)
```

```
            # Auto-loading will handle the rest
        except Exception as e:
            messagebox.showerror("Error", f"Failed to generate IV:
{str(e)}")

    def load_key_from_text(self):
        try:
            key_text = self.key_entry.get(1.0, tk.END).strip()
            # Remove 0x prefix if present (case insensitive)
            if key_text.lower().startswith('0x'):
                key_text = key_text[2:]
            # Filter to only hex characters
            key_text = ''.join(c for c in key_text if c.isdigit() or
c.lower() in 'abcdef')

            if len(key_text) != 32:
                raise ValueError("Key must be exactly 32 hex characters
(128 bits)")

            self.key_bits = hex_string_to_bits(key_text, 128)
            self.key_loaded = True
            self.update_status()
        except Exception as e:
            messagebox.showerror("Error", f"Invalid key: {str(e)}")
```

```
    def load_iv_from_text(self):
        try:
            iv_text = self.iv_entry.get(1.0, tk.END).strip()
            # Remove 0x prefix if present (case insensitive)
            if iv_text.lower().startswith('0x'):
                iv_text = iv_text[2:]
            # Filter to only hex characters
            iv_text = ''.join(c for c in iv_text if c.isdigit() or
c.lower() in 'abcdef')

            if len(iv_text) != 24:
                raise ValueError("IV must be exactly 24 hex characters
(96 bits)")

            self.iv_bits = hex_string_to_bits(iv_text, 96)
            self.iv_loaded = True
            self.update_status()
        except Exception as e:
            messagebox.showerror("Error", f"Invalid IV: {str(e)}")

    def load_key_file(self):
        try:
```

```
            filename = filedialog.askopenfilename(title="Select Key
File")

            if filename:

                self.key_bits = load_hex_bits_from_file(filename, 128)

                # Display in text box using correct bit ordering

                hex_str = bits_to_hex_string(self.key_bits)

                self.key_entry.delete(1.0, tk.END)

                self.key_entry.insert(1.0, hex_str)

                # Auto-loading will handle setting key_loaded and
updating status

                messagebox.showinfo("Success", "Key loaded from file
successfully!")

        except Exception as e:

            messagebox.showerror("Error", f"Failed to load key:
{str(e)}")



    def load_iv_file(self):

        try:

            filename = filedialog.askopenfilename(title="Select IV
File")

            if filename:

                self.iv_bits = load_hex_bits_from_file(filename, 96)

                # Display in text box using correct bit ordering

                hex_str = bits_to_hex_string(self.iv_bits)

                self.iv_entry.delete(1.0, tk.END)
```

```
                self.iv_entry.insert(1.0, hex_str)

                # Auto-loading will handle setting iv_loaded and
updating status

                messagebox.showinfo("Success", "IV loaded from file
successfully!")

        except Exception as e:

            messagebox.showerror("Error", f"Failed to load IV:
{str(e)}")



    def save_key_file(self):

        try:

            if not self.key_loaded:

                messagebox.showerror("Error", "No valid key to save!")

                return



            filename = filedialog.asksaveasfilename(title="Save Key
File", defaultextension=".txt")

            if filename:

                # Convert bits to hex using the same format as the
original file I/O

                hex_bytes = []

                for i in range(0, len(self.key_bits), 8):

                    byte = 0

                    for j in range(8):

                        if i + j < len(self.key_bits):
```

```
                            byte |= (self.key_bits[i + j] << (7 - j))
                    hex_bytes.append(f"{byte:02X}")

                hex_str = ''.join(hex_bytes)
                with open(filename, 'w') as f:
                    f.write(hex_str)
                messagebox.showinfo("Success", "Key saved to file
successfully!")
        except Exception as e:
            messagebox.showerror("Error", f"Failed to save key:
{str(e)}")

    def save_iv_file(self):
        try:
            if not self.iv_loaded:
                messagebox.showerror("Error", "No valid IV to save!")
                return

            filename = filedialog.asksaveasfilename(title="Save IV
File", defaultextension=".txt")
            if filename:
                # Convert bits to hex using the same format as the
original file I/O
                hex_bytes = []
                for i in range(0, len(self.iv_bits), 8):
```

```
                    byte = 0
                    for j in range(8):
                        if i + j < len(self.iv_bits):
                            byte |= (self.iv_bits[i + j] << (7 - j))
                    hex_bytes.append(f"{byte:02X}")

                hex_str = ''.join(hex_bytes)
                with open(filename, 'w') as f:
                    f.write(hex_str)
                messagebox.showinfo("Success", "IV saved to file
successfully!")
        except Exception as e:
            messagebox.showerror("Error", f"Failed to save IV:
{str(e)}")

    def update_status(self):
        if self.key_loaded and self.iv_loaded:
            self.status_label.config(text="Status: Key and IV loaded
successfully", foreground="green")
            self.go_encrypt.pack(pady=8)  # Increased padding
            self.go_decrypt.pack(pady=8)  # Increased padding
        elif self.key_loaded:
            self.status_label.config(text="Status: Key loaded, IV
needed", foreground="orange")
            self.go_encrypt.pack_forget()
```

```
            self.go_decrypt.pack_forget()

        elif self.iv_loaded:

            self.status_label.config(text="Status: IV loaded, Key
needed", foreground="orange")

            self.go_encrypt.pack_forget()

            self.go_decrypt.pack_forget()

        else:

            self.status_label.config(text="Status: Key and IV not
loaded", foreground="red")

            self.go_encrypt.pack_forget()

            self.go_decrypt.pack_forget()



    def load_plaintext_file(self):

        try:

            filename = filedialog.askopenfilename(title="Select
Plaintext File")

            if filename:

                with open(filename, 'r', encoding='utf-8') as f:

                    content = f.read()

                self.plaintext_entry.delete(1.0, tk.END)

                self.plaintext_entry.insert(1.0, content)

                messagebox.showinfo("Success", "Plaintext loaded from
file successfully!")

        except Exception as e:
```

```
            messagebox.showerror("Error", f"Failed to load plaintext:
{str(e)}")



    def encrypt_text(self):

        try:

            if not self.key_loaded or not self.iv_loaded:

                messagebox.showerror("Error", "Please load key and IV
first!")

                return



            # Get plaintext and convert to bits

            plaintext = self.plaintext_entry.get(1.0,
tk.END).rstrip('\n')

            if not plaintext:

                messagebox.showerror("Error", "Please enter plaintext
to encrypt!")

                return



            plaintext_bits = text_to_bits(plaintext)



            # Initialize cipher and encrypt

            self.cipher.initialize(self.key_bits, self.iv_bits)



            # Ask for output filename

            output_file = filedialog.asksaveasfilename(
```

```
                title="Save Encrypted File As",
                defaultextension=".enc",
                filetypes=[("Encrypted files", "*.enc"), ("All files",
"*.*")]
            )

            if output_file:
                cipher_bits = self.cipher.encrypt(plaintext_bits,
output_file)
                messagebox.showinfo("Success", f"Text encrypted and
saved to {output_file}")
        except Exception as e:
            messagebox.showerror("Error", f"Encryption failed:
{str(e)}")

    def encrypt_text(self):
        try:
            if not self.key_loaded or not self.iv_loaded:
                messagebox.showerror("Error", "Please load key and IV
first!")
                return

            # Get plaintext and convert to bits
            plaintext = self.plaintext_entry.get(1.0,
tk.END).rstrip('\n')
            if not plaintext:
```

```
                messagebox.showerror("Error", "Please enter plaintext
to encrypt!")
                return

            plaintext_bits = text_to_bits(plaintext)

            # Initialize cipher and encrypt
            self.cipher.initialize(self.key_bits, self.iv_bits)

            # Ask for output filename
            output_file = filedialog.asksaveasfilename(
                title="Save Encrypted File As",
                defaultextension=".enc",
                filetypes=[("Encrypted files", "*.enc"), ("All files",
"*.*")]
            )

            if output_file:
                cipher_bits = self.cipher.encrypt(plaintext_bits,
output_file)

                # Convert ciphertext bits to hex string for display
                cipher_hex = bits_to_hex_string(cipher_bits)
```

```
                # Determine if we should display the ciphertext in the
text box

                # Use a reasonable threshold - if hex string is longer
than what fits comfortably

                # in the text box (roughly 2000 characters), don't
display it

                max_display_length = 2000



                # Clear the ciphertext display first

                self.ciphertext_display.config(state=tk.NORMAL)

                self.ciphertext_display.delete(1.0, tk.END)



                if len(cipher_hex) <= max_display_length:

                    # Display the ciphertext in the text box

                    self.ciphertext_display.insert(1.0, cipher_hex)

                    self.ciphertext_display.config(state=tk.DISABLED)

                    messagebox.showinfo("Success", f"Text encrypted and
saved to {output_file}\nCiphertext displayed below.")

                else:

                    # Too long to display, just show a message

                    self.ciphertext_display.insert(1.0, f"Ciphertext
too long to display ({len(cipher_hex)} characters)\nSaved to file:
{output_file}")

                    self.ciphertext_display.config(state=tk.DISABLED)

                    messagebox.showinfo("Success", f"Text encrypted and
saved to {output_file}\n(Ciphertext too long to display)")
```

```
        except Exception as e:
            messagebox.showerror("Error", f"Encryption failed:
{str(e)}")

    def encrypt_key(self):
        try:
            if not self.key_loaded:
                messagebox.showerror("Error", "Please load key first!")
                return

            filename = self.encrypt_key_filename.get().strip()
            password = self.encrypt_key_password.get().strip()

            if not filename or not password:
                messagebox.showerror("Error", "Please enter filename
and password!")
                return

            encrypt_key_with_password(self.key_bits, password,
filename)
            messagebox.showinfo("Success", f"Key encrypted and saved to
{filename}")

            # Clear password field
```

```
            self.encrypt_key_password.delete(0, tk.END)


        except Exception as e:
            messagebox.showerror("Error", f"Key encryption failed:
{str(e)}")


    def browse_decrypt_key(self):
        filename = filedialog.askopenfilename(title="Select Encrypted
Key File")
        if filename:
            self.decrypt_key_filename.delete(0, tk.END)
            self.decrypt_key_filename.insert(0, filename)


    def decrypt_key(self):
        try:
            filename = self.decrypt_key_filename.get().strip()
            password = self.decrypt_key_password.get().strip()


            if not filename or not password:
                messagebox.showerror("Error", "Please enter filename
and password!")
                return


            self.key_bits = decrypt_key_with_password(filename,
password)
```

```
            self.key_loaded = True



            # Display decrypted key in setup tab

            hex_str = ''.join(f"{byte:02X}" for byte in
bits_to_bytes(self.key_bits))

            self.key_entry.delete(1.0, tk.END)

            self.key_entry.insert(1.0, hex_str)



            self.update_status()

            messagebox.showinfo("Success", "Key decrypted
successfully!")



            # Clear password field

            self.decrypt_key_password.delete(0, tk.END)

        except Exception as e:

            messagebox.showerror("Error", f"Key decryption failed:
{str(e)}")



    def browse_cipher_file(self):

        filename = filedialog.askopenfilename(

            title="Select Ciphertext File",

            filetypes=[("Encrypted files", "*.enc"), ("All files",
"*.*")]

        )

        if filename:
```

```
            self.cipher_filename.delete(0, tk.END)
            self.cipher_filename.insert(0, filename)

    def decrypt_text(self):
        try:
            if not self.key_loaded or not self.iv_loaded:
                messagebox.showerror("Error", "Please load key and IV
first!")
                return

            cipher_file = self.cipher_filename.get().strip()
            if not cipher_file:
                messagebox.showerror("Error", "Please select a
ciphertext file!")
                return

            # Read ciphertext
            ciphertext_bits = read_ciphertext(cipher_file)
            if not ciphertext_bits:
                messagebox.showerror("Error", "Failed to read
ciphertext file!")
                return

            # Ask for output filename
```

```
            if cipher_file.endswith('.enc'):
                default_output = cipher_file[:-4] + '.dec'
            else:
                default_output = cipher_file + '.dec'

            output_file = filedialog.asksaveasfilename(
                title="Save Decrypted File As",
                initialfile=default_output,
                defaultextension=".dec",
                filetypes=[("Text files", "*.dec"), ("All files",
"*.*")]
            )

            if output_file:
                # Initialize cipher and decrypt
                self.cipher.initialize(self.key_bits, self.iv_bits)
                plaintext_bits = self.cipher.decrypt(ciphertext_bits,
output_file)

                # Display result in text area
                try:
                    with open(output_file, 'r', encoding='utf-8') as f:
                        decrypted_text = f.read()
```

```
                    self.result_text.delete(1.0, tk.END)
                    self.result_text.insert(1.0, decrypted_text)

                    messagebox.showinfo("Success", f"Text decrypted and
saved to {output_file}")
                except Exception as e:
                    messagebox.showinfo("Success", f"Text decrypted and
saved to {output_file}\n(Preview not available: {str(e)})")

        except Exception as e:
            messagebox.showerror("Error", f"Decryption failed:
{str(e)}")

def main():
    root = tk.Tk()
    app = GrainGUI(root)
    root.mainloop()

if __name__ == "__main__":
    main()
```

Appendix B: Marking Rubric

MARKING RUBRIC

Component Title	Part I: Implementation of a Secure Encrypted Storage System using Grain-128AEADv2		Percentage (%)	10%	
Relevant Learning Outcome	Score and Descriptors			Weight (%)	Marks
	Excellent (9-10)	Average (6-8)	Poor (0-5)		
3	▪ The logic in the program is correct/mostly correct. ▪ The implementation is exceptional. All functions work well, handles various inputs accurately. ▪ Well-documented with comments; structured, and free from errors. ▪ Includes a graphical user interface.	▪ Provide more than 50% correct logic in the program. ▪ The system generally works well but may have some minor issues or limitations. ▪ Fairly well documented with some comments, demonstrates decent coding practices. ▪ Includes only a command line interface.	▪ Logic is incorrect in the program. ▪ The system has significant issues that affect its functionality. ▪ The code lacks proper documentation, comments are not included in the code and/or shows errors. ▪ No interface provided, inputs are hardcoded.	10	
Component Title	Part II: Testing and Evaluation of the System		Percentage (%)	5%	
Relevant Learning Outcome	Score and Descriptors			Weight (%)	Marks
	Excellent (4-5)	Average (2-3)	Poor (0-1)		
3	▪ A comprehensive testing is conducted, covering various scenarios. ▪ The testing results are reported in details.	▪ Some testing is conducted covering some relevant scenarios. ▪ The testing results are reported; but may lack some depth or clarity.	▪ Limited/no testing are conducted with/without expected outcome. ▪ Testing results are not reported/ reported vaguely.	5	
Component Title	Part III: Analysis of the Construction of the Underlying Primitive		Percentage (%)	5%	
Relevant Learning Outcome	Score and Descriptors			Weight (%)	Marks
	Excellent (3)	Average (2)	Poor (0-1)		
4 (Q1)	▪ Shows a comprehensive understanding of the reason/rationale of the design choices. ▪ Illustrated with detailed examples.	▪ Shows some understanding of the design rationale but lack depth or clarity in explanations. ▪ Examples could be more detailed, and/or examples are somewhat related but not fully accurate.	▪ Inappropriate report that does not correctly identify the rationale of the design choices. ▪ Limited or no examples, and/or example(s) is not related.	3	
Relevant Learning Outcome	Score and Descriptors			Weight (%)	Marks
	Excellent (2)	Average (1)	Poor (0)		
4 (Q2)	▪ Shows a comprehensive understanding of the rationale of the design choices with respect to the question being asked.	▪ Shows some understanding of the rationale of the design choices with respect to the question being asked.	▪ Inappropriate report that does not correctly identify the rationale of the design choices.	2	
			Total	20	

Note to students: Please include the marking rubric when submitting your coursework.

Appendix C: Turnitin Summary

CYS2309276_Stefanie_Grain128AEAD128_Report.pdf

ORIGINALITY REPORT

19%	9%	6%	15%
SIMILARITY INDEX	INTERNET SOURCES	PUBLICATIONS	STUDENT PAPERS

PRIMARY SOURCES

1	Submitted to Xiamen University Student Paper	5%
2	Submitted to University of Balamand Student Paper	1%
3	Submitted to Universidad Tecnológica de Bolívar,UTB Student Paper	1%
4	Kuldeep Singh Kaswan, Jagjit Singh Dhatterwal, Anand Nayyar. "Digital Personality: A Man Forever - Volume 3: Ontologies to Dialogue Generation", CRC Press, 2025 Publication	1%
5	Submitted to University of Hertfordshire Student Paper	1%
6	Submitted to Polytechnic Institute Australia Student Paper	1%
7	Submitted to University of Northampton Student Paper	1%
8	version.aalto.fi Internet Source	<1%
9	rusmir.eu Internet Source	<1%

Appendix D: Digital Receipt

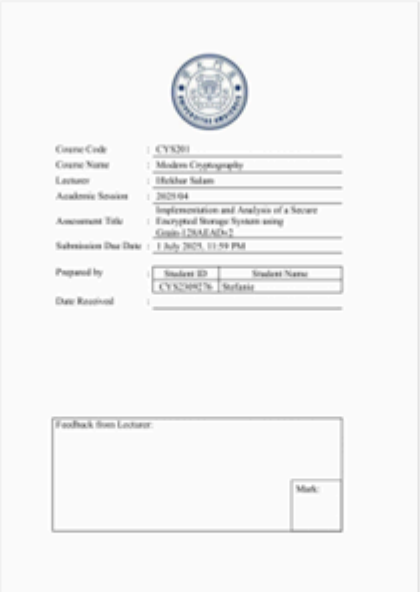
turnitin

Digital Receipt

This receipt acknowledges that Turnitin received your paper. Below you will find the receipt information regarding your submission.

The first page of your submissions is displayed below.

Submission author:	Stefanie -
Assignment title:	Implementation and Analysis of a Secure Encrypted Storage Sy...
Submission title:	CYS2309276_Stefanie_Grain128AEAD128_Report.pdf
File name:	CYS2309276_Stefanie_Grain128AEAD128_Report.pdf
File size:	1.18M
Page count:	82
Word count:	9,569
Character count:	54,140
Submission date:	30-Jun-2025 10:26PM (UTC+0800)
Submission ID:	2708386429



Copyright 2025 Turnitin. All rights reserved.